

Pipeline Parallelism for Block Attention Residuals: Design ideas and motivations

0. One page

```
Block AttnRes: every layer attends over ALL earlier blocks + the running partial block
|
v  split by layers into pipeline stages
R1 the block stack must cross every stage boundary with the hidden state
R2 the final aggregation (output_res_proj -> output_res_norm) runs only on the stage that
owns lm_head
R3 the stack grows with depth; a boundary inside a block puts a partial block on the wire
|
+-- naive transport: send the whole stack every hop -> correct, bytes grow with the
stage index
+-- cross-stage cache (report 4.1): V virtual stages per rank; a hop carries only the
delta the receiver lacks
|
v  three new dependencies
(a) sender and receiver must agree on what a hop carries -> routing tables
(BlockLayoutTables), no metadata on the wire
(b) the cache is per micro-batch -> a key that survives P2P
(the schedule's chunk id)
(c) a cached block's gradient must return from every later stage to the stage that
committed it -> two gradient routes
|
v
torch's PipelineStage knows one protocol: outputs to the next stage, gradients from
it -> section 3
```

1. What PP / VP demand of the model and the micro-batches

requirement	why	where
stack crosses every boundary	each layer attends over all earlier blocks	hop payload (hidden_TD, delta_TND); the model takes and returns the whole stack and knows nothing of transport
aggregation only on the head stage	one aggregation over the whole stack	kimik3_module_fqns_per_model_part keeps the res modules with lm_head
partial blocks on the wire	a block's first layer joins the stack before it attends	first_layer_in_block; a boundary at a block start needs nothing special

Block size and stage count need not divide: K3 is 93 layers = 7 x 12 + 9, the debug flavor uses 33 so no pipeline shape divides it, and routing is built from the actual split (one `all_gather_object` of layer -> stage).

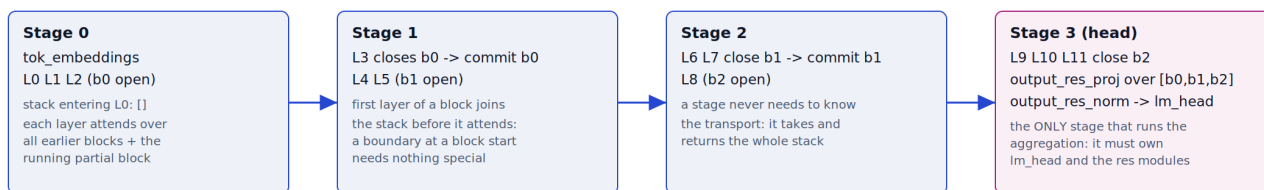
Two transports share that table behind one flag (`attn_res_cache`), and they are the A/B of the results tables: naive sends the whole stack every hop (correct, no adapter, what plain 1F1B runs); the cached delta keeps a block committed at stage S on the wire for P-1 hops, after which every receiving rank already holds it from its virtual stage S-P, so the per-hop payload is bounded by the last P-1 stages' commits and does not grow with depth.

Micro-batch dependencies: the cache key must be the schedule's integer chunk id (`id(tensor)` fails, NCCL hands out fresh receive buffers); blocks are released after the rank's LAST virtual stage forward for that micro-batch; Interleaved1F1B needs micro-batches $\geq$ virtual stages and virtual stages divisible by `pp`. PP is bitwise with no-PP at one micro-batch and differs by a bf16 accumulation-order

margin at many (step 1: $\sim 4e-4$ on the loss, 1.3% on grad_norm); VP adds nothing (VP=1 and VP=4 bitwise).

Block AttnRes under a pipeline split: the stack crosses every boundary, partial blocks ride the wire, the head stage aggregates

Drawing: 12 layers, blocks of 4 (b0 = L0-3, b1 = L4-7, b2 = L8-11), 4 stages of 3 layers -- every block boundary falls inside a stage.



Payload of each hop: (hidden_TD, stack_TND) -- the stack grows with depth; a block cut by the boundary is a partial column on the wire



Two transports of the same stack

naive: every hop carries the whole stack -> correct, no adapter (plain 1F1B, one stage per rank), bytes per hop grow with the stage index
cross-stage cache (report 4.1): with V virtual stages per rank, a hop carries only the blocks the receiving rank has not seen;
a block committed at stage S is fresh on the wire for P-1 hops, then every receiving rank holds it from its virtual stage S-P
=> per-hop payload bounded by the commits of the last P-1 stages, independent of depth; both transports are one routing table (attn_res_cache)

Figure 1. 12 layers, blocks of 4, 4 stages: the payload per hop, and the two transports.

2. The design: what each problem forced

The published note from the model's infrastructure team puts an adapter after the pipeline communication: it concatenates the received block with the cached ones, its backward accumulates all blocks' gradients and sends the buffer on, and the send/rcv hides in interleaved steady state. It is a statement about wire bytes, and it leaves three problems to whoever implements it.

Who holds what, without metadata on the wire. A hop's payload depends on what the receiver already has, so both sides must agree on the columns before the first send. The layout tables are built once from the actual layer -> stage split and simulated offline on both sides; nothing about the payload travels with it.

A micro-batch key that survives P2P. The cache is per micro-batch, and tensor identity does not survive the transport (fresh receive buffers). The key has to come from the schedule. Subclassing the stage makes this trivial -- the chunk id is the argument of the call being overridden -- where anything outside the stage has to reach for it.

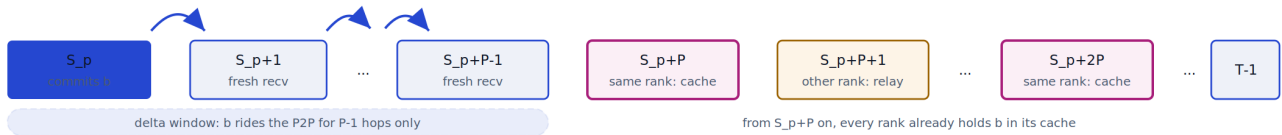
The backward of a cached block. A block committed at one stage is read by several later stages, so its gradient has to come back from all of them to the stage that committed it. Every general mechanism we tried failed for a reason worth stating:

1. A custom collective inside `autograd.Function.backward` deadlocks: the autograd engine is single-threaded and depth-first, the peer has not reached the matching Function, and the exchange races the schedule's own backward P2P.
2. Flushing the gradients after each micro-batch's backward deadlocks too: under interleaving, ranks reach a given micro-batch's backward at different times, so the P2P has no peer.
3. One batched exchange at step end removes the deadlock but keeps every micro-batch's graph alive until the step ends.
4. Letting the gradients ride the schedule's own backward P2P is correct across ranks and needs no new communication. It fails in exactly one case: a block a rank commits at one virtual stage and reads back from its own store at the next, where the consumer's backward walks into the producer's graph and frees it a second time.
5. `retain_graph=True` fixes that case and gives up the memory PP exists to save; the cost grows with virtual stages and micro-batches.

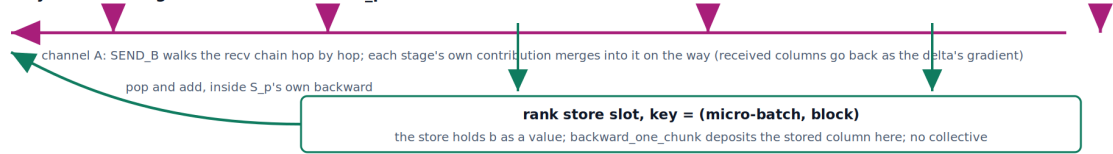
6. Wrapping the same-rank hand-off in autograd Functions still double-fires when both ends land in one micro-batch's backward.

What works is to stop treating the cached block as a graph at all. The store holds VALUES, so a consumer's input has no upstream graph and every forward graph is traversed once per micro-batch; the gradient is deposited in a slot keyed by (micro-batch, producing stage, commit index) and collected by the stage that brought the block onto the rank, before its own backward -- an ordering every schedule already provides, since it runs the later virtual stages' backward first. Across ranks nothing is added: the gradient rides the pipeline's own backward P2P. The tables say how many deposits each block must receive, so a lost gradient raises instead of training silently.

Forward: where block b (committed by stage S_p , rank R_p) is read from



Backward: how every consumer's gradient for b reaches S_p



At S_p , before its own backward (`_retrieve_recv_grads`):

grad arriving on channel A + sum of the slot deposits (channel B) = the block's full gradient, then S_p 's forward graph is traversed exactly once

Counts for a block from S_p , $v_p = S_p \div P$

consumers in total = $T - 1 - S_p$

channel B (same rank) = $V - 1 - v_p$

channel A (everything else) = $(T - 1 - S_p) - (V - 1 - v_p)$

Self-check

`layout.deposits_expected(b,  $S_p$ )` returns $V - 1 - v_p$ offline.

`_collect_into` compares the deposits it pops against it and raises

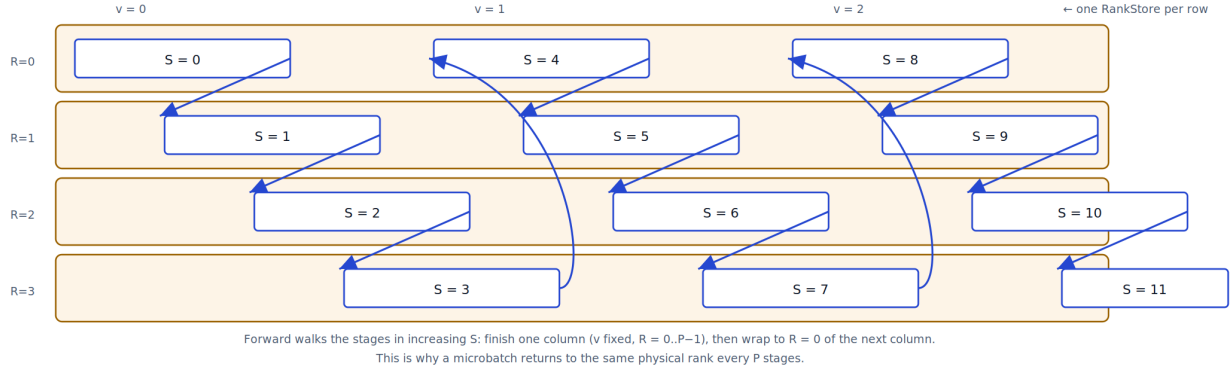
on a mismatch: a missing capture is a lost gradient no loss curve shows.

Figure 2. The delta window (a block is fresh on the wire for P-1 hops) and the two gradient routes: the pipeline's own backward P2P across ranks, a store slot within a rank.

The transport is an argument of the pipelining entry rather than an environment variable (a non-uniform export once gave ranks different topologies and hung), the layout comes from the model config, and the split is whatever the recipe asks for: an `all_gather_object` of layer -> stage replaces the even-split assumption, so an irregular model needs no divisibility.

The stage itself is a `PipelineStage` subclass. `forward_one_chunk` assembles the stack from the rank's store plus the received delta, runs the stage, keeps its commits and sends only what the next rank lacks; `backward_one_chunk` reads the gradient of the assembled stack -- a leaf the stage owns -- returns the received columns as the delta's gradient and deposits the stored columns; `_retrieve_recv_grads` collects the deposits for the blocks this rank brought in, before their producer's backward. No hooks, no custom Functions, no detach tricks: the schedule's own ordering carries the design. One core hook makes it possible, `pipeline_llm(..., stage_class=...)`. Step 1 is bitwise with a single GPU on every $pp \times vp$ cell of the irregular debug model, from 2 to 32 stages, under both transports.

V columns × P rows (shown: P = 4, V = 3 → T = 12 global stages)

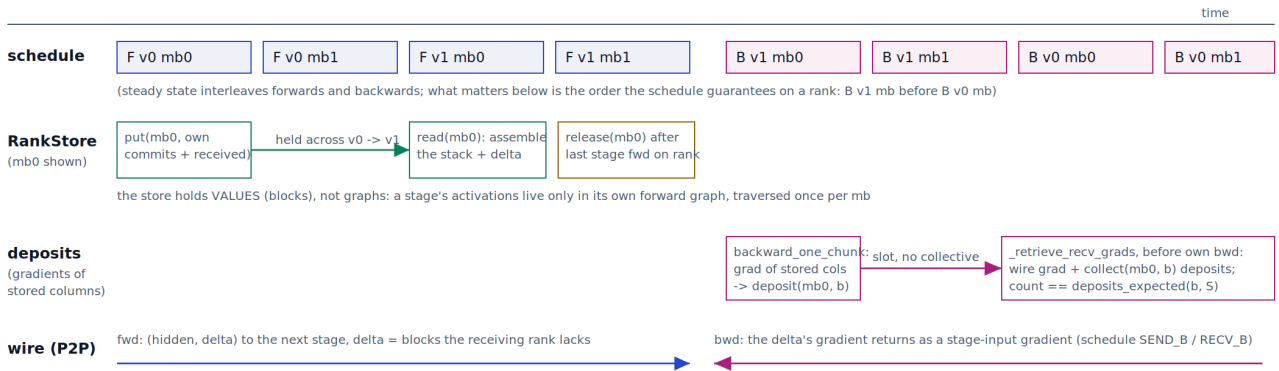


Key corollary — when rank R runs stage S, its cache already holds every block committed at stages $\leq S - P$,
because the rank's previous virtual stage was exactly $S - P$. The delta payload and both gradient channels are consequences of this one fact.

Figure 3. The looped assignment $S = v * P + R$ (shown: $P=4, V=3$). A micro-batch walks the stages in increasing S, so it returns to the same rank every P stages and that rank's store already holds every block committed at stages $\leq S - P$. The delta window, the same-rank slot and its expected deposit count all follow from this.

One rank R under Interleaved1F1B: virtual stages $v_0 = S_R$ and $v_1 = S_R + P$, micro-batches mb0 and mb1

Time runs left to right. The rank's RankStore is shared by v_0 and v_1 ; every entry is keyed by (micro-batch, block).



What the timeline fixes, and what it needs

1. micro-batch key: the store is per mb, so forward and backward must know the mb -- the schedule's chunk id (fwd_chunk_id / bwd_chunk_id), never id(tensor)
 2. release: only after the rank's LAST virtual stage forward for that mb (v_1 reads what v_0 kept); dropping at v_0 's backward would lose v_1 's read
 3. two gradient routes for one block: across ranks the delta's gradient rides the schedule's backward P2P; on a rank the stored column's gradient is a deposit that the stage which brought the block onto the rank collects before its own backward -- ordered by the schedule itself
 4. counts: deposits_expected(b, S) is a static function of the routing tables; a missing deposit raises instead of training quietly
 5. constraints: micro-batches $\geq$ virtual stages; virtual stages divisible by P; PP exact at one mb, a bf16 accumulation-order margin at many, VP adds nothing
- Subclass version (pipeline_stage.py): no tensor hook, no autograd Function, no detach trick -- the store, the deposits and the schedule order are the whole mechanism.

Figure 4. One rank, virtual stages v_0 / v_1 , micro-batch 0: put at v_0 's forward, read at v_1 's forward, release after the rank's last virtual stage forward; deposit at v_1 's backward, collect before v_0 's backward, ordered by the schedule.

symptom	cause	fix
every middle stage missed its cache	id(tensor) as the key; NCCL fresh buffers	the schedule's chunk id
NCCL watchdog timeout	custom P2P inside autograd / in a flush	the gradients ride the schedule's own backward P2P
"backward through the graph a second time"	a same-rank cached block still attached to the producer's graph	the store holds values, not graphs
memory grows with virtual stages and micro-batches	retain_graph=True as a stop-gap for the same-rank re-read	values in the store, deposits collected by the schedule's ordering
ranks with different topologies hang	the transport chosen by an environment variable, exported non-uniformly	an argument of the pipelining entry

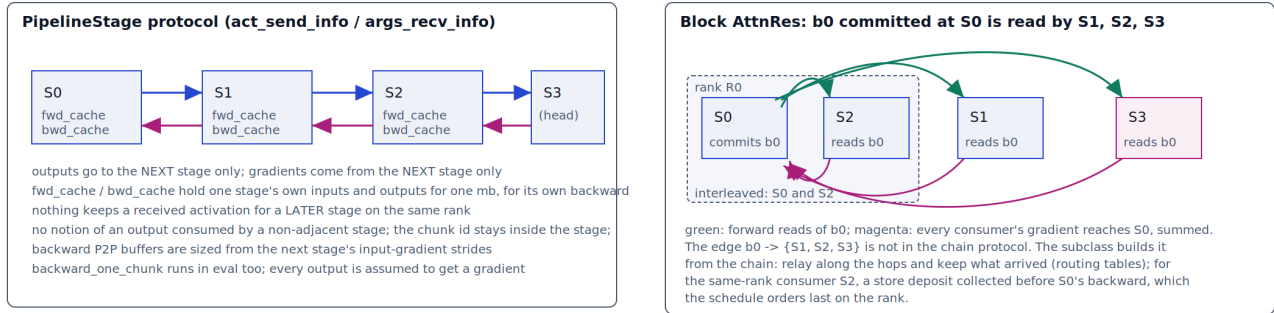
symptom	cause	fix
even split only	interleaved ranks lack the global layer -> stage map	one <code>all_gather_object</code>
Tensors for P2P must be non-overlapping and dense	torch sizes the backward receive buffer from the next stage's input-gradient strides; a stage starting with cat / stack / slice yields view gradients	model-side <code>_DenseGradient</code> (identity, <code>.contiguous()</code> backward); upstream should allocate <code>torch.empty(shape)</code> and send <code>.contiguous()</code>
"no gradient arrived for the payload" under LoRA	nothing trainable upstream of the payload (frozen embedding feeds block 0); the next stage has no gradient channel	<code>_retrieve_recv_grads</code> accepts <code>None</code> ; deposits discarded
<code>KeyError: 0</code> in verl's forward-only log-prob pass	<code>schedule.eval</code> calls <code>backward_one_chunk</code> with backward off; the base returns, the override read the cache it never filled	honour <code>has_backward</code> , drop the chunk's bookkeeping
<code>KeyError</code> in the HF initial load on a stage	layer-0 placeholders were shaped from layer 1, absent on that stage	place only on the rank holding layer 0, shape from the config
'FSDPParam' has no attribute '_unsharded_param' on a text batch under PP	FSDP2's root final callback runs <code>post_backward</code> for every group, and the no-reduce branch (PP's non-last micro-batches) lacks the <code>hasattr</code> guard the reduce branch has; a never-forwarded FSDP unit (the vision tower on text) has no <code>_unsharded_param</code>	a text-only model has no tower (the text alias sets <code>vision_encoder=None</code>); the torch guard is an upstream issue, not a parallelism change
step 10 spreads by percents	Adam's first step is <code>lr * sign(g)</code> ; bf16 rounding flips 0.2% of elements and the descent amplifies it; fp32 total norm does not remove it	step 1 and per-parameter gradients are the bar

3. What `torch.distributed.pipelining` lacks, and the suggestions

`PipelineStage` describes a chain (`act_send_info` / `args_recv_info`: outputs to the next stage, gradients from it), and its `fwd_cache` / `bwd_cache` hold one stage's own inputs and outputs for one micro-batch, for its own backward. Nothing keeps a received activation for a later stage on the same rank, and nothing routes a gradient to a non-adjacent producer. Block AttnRes needs a multi-consumer edge. In order of value:

- Multi-consumer stage outputs.** The library owns the routing table (`BlockLayoutTables` is that table, computed outside it) and decides per hop what travels and what is kept; the chain relay with a rank store is one implementation, bounded to the last P-1 stages' commits.
- Rank-local delivery for same-rank consumers.** The schedule knows `stage_index_to_group_rank`; such an output can be handed over by reference and its gradient merged before the producer's backward, which the schedule already orders last on the rank (route B without a hook; the subclass proves the ordering is enough).
- Chunk id and lifecycle for the submodule.** A context with the current micro-batch plus callbacks at micro-batch end and step end; removes the two wrappers and enables per-micro-batch release.
- Dense backward P2P buffers.** `torch.empty(shape)` receive and `.contiguous()` before send; stride-sized buffers fail when a stage starts with a view op.
- Forward-only and no-gradient contracts.** `backward_one_chunk` under `has_backward=False` as part of the subclass contract; stage outputs with `requires_grad=False` get `None` rather than an assumed gradient.

torch.distributed.pipelining today: a chain. Block AttnRes: one output with many consumers. Five interfaces would close the gap.



The five interfaces, where each one sits

- 1 multi-consumer stage output: the library owns the routing table (per stage: commits, held on rank, carried by the hop); the relay + store is one implementation
- 2 rank-local delivery: a consumer on the producer's rank (interleaved) gets the tensor by reference; the schedule merges its gradient before the producer's backward
- 3 chunk id and lifecycle for the submodule: a context (current mb) plus callbacks at mb end and step end -- removes the wrappers, enables per-mb release
- 4 dense backward P2P buffers: empty(shape) receive + contiguous() send; stride-sized buffers fail when a stage starts with cat / stack / slice
- 5 forward-only and no-gradient contracts: backward_one_chunk under has_backward=False as part of the subclass contract; outputs with requires_grad=False get None

Where they bite today: 1 and 2 are the whole adapter / subclass (BlockLayoutTables, RankStore, deposits); 3 is the two wrappers the first version needed;
4 is the _DenseGradient identity op in the model; 5 is the has_backward guard and the None-gradient payload under LoRA (2026-09-06).

torch's own fwd_cache / bwd_cache are not 1 or 2: they hold a stage's own inputs and outputs for its own backward, nothing across stages or ranks.

Figure 5. Left: the chain protocol and its per-stage caches. Right: b0 committed at S0 read by S1 / S2 / S3, with S2 on S0's rank. The five suggestions where each bites.